

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

Exp. No: 3

Date: 24-07-2026

Name: Kushaal Shyam Potta

Reg. No: 3122235001071

Implementation of Recursive Decent Parser

Aim

To write a recursive descent parser for the given grammar, after removing its left recursion.

Grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

$F \rightarrow (E) \mid id$

Tools Used

- GCC

Code (rdp.c)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
const char *input;
```

```
int i = 0;
```

```
int error = 0;
```

```
void E();
```

```
void E_prime();
```

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```
void T();
```

```
void T_prime();
```

```
void F();
```

```
void match(char expected) {
```

```
    if (input[i] == expected) {
```

```
        i++;
```

```
    } else {
```

```
        error = 1;
```

```
    }
```

```
}
```

```
void match_id() {
```

```
    if (input[i] == 'i' && input[i + 1] == 'd') {
```

```
        i += 2;
```

```
    } else {
```

```
        error = 1;
```

```
    }
```

```
}
```

```
void E() {
```

```
    T();
```

```
    E_prime();
```

```
}
```

```
void E_prime() {
```

```
    if (input[i] == '+') {
```

```
        match('+');
```

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```

    T();
    E_prime();
}
}

void T() {
    F();
    T_prime();
}

void T_prime() {
    if (input[i] == '*') {
        match('*');
        F();
        T_prime();
    }
}

void F() {
    if (input[i] == '(') {
        match('(');
        E();
        match(')');
    } else if (input[i] == 'i' && input[i + 1] == 'd') {
        match_id();
    } else {
        error = 1;
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

}

```
void parse_string(const char *str) {
    input = str;
    i = 0;
    error = 0;
    printf("Parsing string: \"%s\\n\", input);

    E();
    if (error == 0 && input[i] == '\\0') {
        printf("Result: SUCCESS (String successfully parsed!)\n\\n");
    } else {
        printf("Result: REJECTED (Syntax Error detected!)\n\\n");
    }
}
```

```
int main() {
    parse_string("id+id*id");
    parse_string("(id+id*id");
    parse_string("id-id");
    return 0;
}
```

Output:

Course Code	UCS2702	Course Name	Compiler Design	
Regulation	2021	2023-27	Academic Year	2026-27 (ODD)

```

● kushaal@Kushaals-MacBook-Air Compiler-Design % cd "Assignment 3"
● kushaal@Kushaals-MacBook-Air Assignment 3 % gcc rec.c -o rec
● kushaal@Kushaals-MacBook-Air Assignment 3 % ./rec
Parsing string: "id+id*id"
Result: SUCCESS (String successfully parsed!)

Parsing string: "(id+id*id"
Result: REJECTED (Syntax Error detected!)

Parsing string: "id-id"
Result: REJECTED (Syntax Error detected!)

```

Learning Outcomes

- Learnt to turn each grammar rule into a recursive function so that top-down parsing is carried out in C.
- Learnt to handle operator precedence, match tokens, and detect syntax errors while the program runs.